



Contents

hide

Beginning

- 1 Overview
- > 2 Fancontrol (lm-sensors)
- > 3 NBFC
- > 4 Dell laptops
- > 5 ThinkPad laptops
 - 5.1 Installation
 - 5.2 Running
- 6 Lenovo Legions laptops
- > 7 ASUS laptops
- > 8 AMDGPU sysfs fan control
- > 9 Troubleshooting

Fan speed control

🌐 3 languages

Page Discussion Read View source View history Tools

Fan control can bring various benefits to your system, such as quieter working system and power saving by completely stopping fans on low CPU load.

Warning: Configuring or completely stopping fans on high system load might result in permanently damaged hardware, or thermal throttling at best.

Related articles

[lm_sensors](#)

[Undervolting CPU](#)

[CPU frequency scaling](#)

1 Overview



This article or section needs language, wiki syntax or style improvements. See [Help:Style](#) for reference.

Reason: The note should be merged with the introduction, the tools should be left to their respective sections if they have one, while the remaining ones should be put at the bottom in a "catch-all" section (with a proper title?). (Discuss in [Talk:Fan speed control](#))

Note: Laptop users should be aware about how cooling system works in their hardware. Some laptops have single fan for both CPU and GPU and cools both at the same time. Some laptops have two fans for CPU and GPU, but the first fan cools down CPU and GPU at the same time, while the other one cools CPU only. In some cases, you will not be able to use the **Fancontrol** script due to incompatible cooling architecture (e.g. one fan for both GPU and CPU). See [\[1\]](#) for some more information about this topic.

There are multiple working solutions for fan control for both desktops and notebooks. Depending on your needs:

- Fancontrol (lm-sensors)** — Script (written in Bash) to configure fan speeds. Most suitable for desktops and laptops, where fan controls are available via [sysfs \(5\)](#).
- NoteBook Fan Control (NBFC)** — Cross-platform solution for laptop fan control, written in C# and works under **Mono** runtime. Most suitable for latest, unsupported by **Fancontrol** laptops.
- NBFC-Linux** — A lightweight implementation of NBFC, written in C. It does not depend on the Mono framework. It can be installed as [nbfc-linux^{AUR}](#) or [nbfc-linux-git^{AUR}](#).
- Dell laptops** — Alternative fan control daemon for some Dell laptops.
- ThinkPad laptops** — Fan configuration for some ThinkPad laptops.
- Lenovo Legion Linux** — Fan speed configuration and other settings for some Lenovo Legion laptops.
- ASUS laptops** — Configure some ASUS laptops for **Fancontrol** or manual control.
- [fan2go-git^{AUR}](#) — An alternative to Fancontrol independent of device-paths.
- [MControlCenter](#) — Fan control application for MSI laptops.
- [fw-ectool-git^{AUR}](#) — Fan configuration for **Framework Laptops**.
- [CoolerControl^{AUR}](#) — A fan control daemon with GUI for *sysfs* and *liquidctl* devices.

2 Fancontrol (lm-sensors)

`fancontrol` is a part of [lm_sensors](#), which can be used to control the speed of CPU/case fans.

Support for newer motherboards may not yet be in the Linux kernel. Check the [lm-sensors devices \(archive\)](#) table to see if experimental drivers are available for such motherboards.

2.1 lm-sensors

The first thing to do is to run

```
# sensors-detect
```

This will detect all of the sensors present and they will be used for *fancontrol*. After that, run the following to check if it detected the sensors correctly:

```
$ sensors

coretemp-isa-0000
Adapter: ISA adapter
Core 0:      +29.0°C  (high = +76.0°C, crit = +100.0°C)
...
it8718-isa-0290
Adapter: ISA adapter
Vcc:          +1.14 V  (min = +0.00 V, max = +4.08 V)
VTT:          +2.08 V  (min = +0.00 V, max = +4.08 V)
+3.3V:        +3.33 V  (min = +0.00 V, max = +4.08 V)
NB Vcore:     +0.03 V  (min = +0.00 V, max = +4.08 V)
VDRAM:        +2.13 V  (min = +0.00 V, max = +4.08 V)
fan1:         690 RPM  (min =    10 RPM)
temp1:        +37.5°C  (low  = +129.5°C, high = +129.5°C)  sensor = thermistor
temp2:        +25.0°C  (low  = +127.0°C, high = +127.0°C)  sensor = thermal diode
```

Note: If the output does not display an RPM value for the CPU fan, one may need to [#increase the fan divisor for sensors](#). If the fan speed is shown and higher than 0, this is fine.

2.2 Configuration

Once the sensors are properly configured, use [pwmconfig \(8\)](#) to test and configure fan speed control. Following the guide should create `/etc/fancontrol`, a customized configuration file. In the guide, the default answers are in parenthesis if you press enter without typing anything. Enter `y` for yes, `n` for no.

```
# pwmconfig
```

Note: Some users may experience issues when using `/sys/class/hwmon/` paths for their configuration file. *hwmon* class device symlinks point to the absolute paths, and are used to group all of the *hwmon* sensors together into one directory for easier access. Sometimes, the order of the *hwmon* devices change from a reboot, causing *fancontrol* to stop working. See [#Device paths have changed in /etc/fancontrol](#) for more information on how to fix this.

2.2.1 Tweaking

Some users may want to manually tweak the configuration file after running `pwmconfig` with root privileges, usually to fix something. For manually tweaking the `/etc/fancontrol` configuration file, see [fancontrol \(8\)](#) for the variable definitions.

Users will probably encounter the *hwmon* path issues as noted above in [#Fancontrol \(lm-sensors\)](#). See [#Device paths have changed in /etc/fancontrol](#) for more information.

Tip: Use `MAXPWM` and `MINPWM` options that limit fan speed range. See [fancontrol \(8\)](#) for details.

Note: Temperature and fan sensor paths could change as well (usually on a kernel update) (e.g. `hwmon0/device/temp1_input` becomes `hwmon0/temp1_input`). Check the `fancontrol.service` [unit status](#) to find out which path is the troublemaker and correct your configuration file accordingly.

2.3 Running Fancontrol

Try to run *fancontrol*:

```
# fancontrol
```

A properly configured setup will not output errors and will take control of the system fans. Users should hear system fans starting shortly after executing this command. *fancontrol* can also be run by [starting/enabling](#) `fancontrol.service`.

For an unofficial GUI, [install fancontrol-gui^{AUR}](#) or [fancontrol-kcm^{AUR}](#)^{[[broken link](#): package not found]}.

2.4 Fancontrol stops working after suspend–wake cycles

Unfortunately, *fancontrol* does not work after suspending. As per the [filed bug](#), you will have to restart *fancontrol* after suspending. This can be achieved automatically by a [systemd hook](#).

3 NBFC

Note: NBFC has been unmaintained since Mar 29, 2020. New user configs can still be created manually, however predefined configurations have not been added since that time. There are forks that exist to add new configs, such as [nbfc-revive](#).

NBFC is a cross-platform fan control solution for notebooks. It comes with a powerful configuration system, which allows to adjust it to many different notebook models, including some of the latest ones.

3.1 Installation

NBFC can be installed as `nbfc`^{AUR} or `nbfc-git`^{AUR}. Also `start/enable` `nbfc.service`.

3.2 Configuration

NBFC comes with pre-made profiles. You can find them in `/opt/nbfc/Configs/` directory. When applying them, use the exact profile name without a file extension (e.g. `some_profile.xml` becomes `"some_profile"`).

Check if there is anything NBFC can recommend:

```
$ nbfc config -r
```

If there is at least one model, try to apply this profile and see how fan speeds are being handled. For example:

```
$ nbfc config -a "Asus Zenbook UX430UA"
```

Note: If you are getting `File Descriptor does not support writing`, `delete` `StagWare.Plugins.ECSysLinux.dll` [\[2\]](#)[🔗] and **restart** `nbfc.service`:

```
# mv /opt/nbfc/Plugins/StagWare.Plugins.ECSysLinux.dll /opt/nbfc/Plugins/StagWare.Plugins.ECSysLinux.dll.ol
d
```

If above solution did not help, try appending `ec_sys.write_support=1` to **kernel parameters**.

If there are no recommended models, go to [NBFC git repository](#)[🔗] or `/opt/nbfc/Configs/` and check if there are any similar models available from the same manufacturer. For example, on **Asus Zenbook UX430UQ**, the configuration **Asus Zenbook UX430UA** did not work well (fans completely stopped all the time), but **Asus Zenbook UX410UQ** worked fantastically.

Run `nbfc` to see all options. More information about configuration is available at [upstream wiki](#)[🔗].

4 Dell laptops

i8kutils is a daemon to configure fan speed according to CPU temperatures on some Dell Inspiron and Latitude laptops. It uses the `/proc/i8k` interface provided by the `i8k` driver (an alias for `dell_smm_hwmon`). Results will vary depending on the exact model of laptop.

If fancontrol will not work on your system, use the `ignore_dmi=1` **kernel module parameter** to load `dell_smm_hwmon`.

Warning: *i8kutils* BIOS system calls stop the kernel for a moment on some systems (confirmed on Dell 9560), this can lead to side effects like audio dropouts, see https://bugzilla.kernel.org/show_bug.cgi?id=201097[🔗]

4.1 Installation

`i8kutils`^{AUR} is the main package to control fan speed. Additionally, you might want to install these:

- `acpi` — must be installed to use *i8kmon*.
- `tc1` — must be installed in order to run *i8kmon* as a background service (using the `--daemon` option).
- `tk` — must be installed together with `tc1` to run as X11 desktop applet.
- `dell-bios-fan-control-git`^{AUR} — recommended if your BIOS overrides fan control.

4.2 Configuration

The temperature points at which the fan changes speed can be adjusted in the configuration file `/etc/i8kutils/i8kmon.conf`. Only three fans speeds are supported (high, low, and off). Look for a section similar to the following:

```
set config(0)  {{0 0}  -1  55  -1  55}
set config(1)  {{1 1}  45  75  45  75}
set config(2)  {{2 2}   65 128  65 128}
```

This example starts the fan at low speed when the CPU temperature reaches 55 °C, switching to high speed at 75 °C. The fan will switch back to low speed once the temperature drops to 65 °C, and turns off completely at 45 °C.

Tip: If when running `i8kmon` with the verbose option you notice that the state changes (example of an output: `# (57>=55), state=1, low=45, high=75`) but right and left fans report state 0, you might consider changing the speed value of the first state in the configuration file from default 1000 to 2000 or higher.

4.3 Installation as a service

i8kmon can be started automatically by `starting/enabling` `i8kmon.service`.

4.4 BIOS overriding fan control

Some newer laptops have BIOS fan control in place which will override the OS level fan control. To test if this the case, run `i8kmon` with verbose mode in a command line, make sure the CPU is idle, then see if the fan is turned off or turned down accordingly.

If the BIOS fan control is in place, you can try using `dell-bios-fan-control-git`^{AUR}:

Warning: Turning off BIOS fan control could result in damage to your hardware. Make sure you have *i8kmon* properly set up beforehand, or leave the CPU idle while you test this program.

To enable BIOS fan control:

```
# dell-bios-fan-control 1
```

To disable BIOS fan control:

```
# dell-bios-fan-control 0
```

BIOS fan control can be automatically disabled by `starting/enabling` `dell-bios-fan-control.service`.

5 ThinkPad laptops

Some fan control daemons include `simpfand-git`^{AUR} and `thinkfan`^{AUR} (recommended).

5.1 Installation

Install `thinkfan`^{AUR} or `thinkfan-git`^{AUR}. Optionally, but recommended, install `lm_sensors`. If needed, a GUI is available with `thinkfan-ui`^{AUR}. Then have a look at the files:

```
# pacman -Ql thinkfan
```

Note that the *thinkfan* package installs `/usr/lib/modprobe.d/thinkpad_acpi.conf`, which contains the following **kernel module parameter**:

```
options thinkpad_acpi fan_control=1
```

Note: New Thinkpad models may require an additional `experimental=1` kernel module parameter. So, it is important to check fan functionality.

So fan control is enabled by default.

Now, load the module:

```
# modprobe thinkpad_acpi
# cat /proc/acpi/ibm/fan
```

You should see that the fan level is "auto" by default, but you can echo a level command to the same file to control the fan speed manually:

```
# echo level 1 > /proc/acpi/ibm/fan
```

Fan Levels	
Level	Effect
0	off
2	low speed
4	medium speed
7	maximum speed
If fan control is disabled by BIOS, you can still use the BIOS	

auto	default - automatic, the fan RPM is controlled by the BIOS
full-speed / disengaged	the maximum fan speed; here the controller does not monitor the fan speed

The *thinkfan* daemon will do this automatically.

"7" is not the same as "disengaged". "7" is the maximum regulated speed (corresponds to "full-speed"). disengaged is the maximum unregulated speed. See [ThinkWiki](#) for more details.

Finally, **enable** the `thinkfan.service`.

To configure the temperature thresholds, you will need to copy the example configuration file (`/usr/share/doc/thinkfan/examples/thinkfan.yaml`) to `/etc/thinkfan.conf`, and modify to taste. This file specifies which sensors to read, and which interface to use to control the fan. Some systems have `/proc/acpi/ibm/fan` and `/proc/acpi/ibm/thermal` available; on others, you will need to specify something like:

```
hwmon: /sys/devices/virtual/thermal/thermal_zone0/temp
```

to use generic *hwmon* sensors instead of thinkpad-specific ones.

A configuration example can be found in [Gentoo:Fan speed control/thinkfan#Configuration](#).

5.2 Running

You can test your configuration first by running thinkfan manually (as root):

```
# thinkfan -n
```

and see how it reacts to the load level of whatever other programs you have running.

When you have it configured correctly, **start/enable** `thinkfan.service`.

6 Lenovo Legions laptops

The tool [Lenovo Legion Linux](#) allows to change the fan curves that are stored in the embedded controller. It consists of a kernel module that must be compiled and loaded. Currently, there is no package, but it must be compiled and installed from source.

Then the fan curve can be set via the hwmon interface. This can be done with the provided script or the Python GUI.

7 ASUS laptops

This topic will cover drivers configuration on ASUS laptops for [Fancontrol \(lm-sensors\)](#).

7.1 Kernel modules

In configuration files, we are going to use full paths to *sysfs* files (e.g. `/sys/devices/platform/asus-nb-wmi/hwmon/hwmon[[:print:]]*/pwm1`). This is because `hwmon1` might change to any other number after reboot. [Fancontrol \(lm-sensors\)](#) is written in **Bash**, so using these paths in configuration file is completely acceptable. You can find complete `/etc/fancontrol` configuration file examples at [ASUS N550JV#Fan control](#).

7.1.1 asus-nb-wmi

`asus-nb-wmi` is a kernel module, which is included in the Linux kernel and is loaded automatically on ASUS laptops. It will only allow to control a single fan and if there is a second fan you will not have any controls over it. Note that blacklisting this module will prevent keyboard backlight to work.

Below are the commands to control it. Check if you have any controls over your fan:

```
# echo 255 > /sys/devices/platform/asus-nb-wmi/hwmon/hwmon[[:print:]]*/pwm1      # Full fan speed (Value: 255)
# echo 0 > /sys/devices/platform/asus-nb-wmi/hwmon/hwmon[[:print:]]*/pwm1      # Fan is stopped (Value: 0)
# echo 2 > /sys/devices/platform/asus-nb-wmi/hwmon/hwmon[[:print:]]*/pwm1_enable  # Change fan mode to automatic
# echo 1 > /sys/devices/platform/asus-nb-wmi/hwmon/hwmon[[:print:]]*/pwm1_enable  # Change fan mode to manual
# echo 0 > /sys/devices/platform/asus-nb-wmi/hwmon/hwmon[[:print:]]*/pwm1_enable  # Change fan mode to full speed
```

If you were able to modify fan speed with above commands, then continue with [#Generate configuration file with pwmconfig](#).

7.1.2 asus_fan

`asus_fan` is a kernel module, which allows to control both fans on some older ASUS laptops. It does not work with the most recent models.

Install the **DKMS `asus-fan-dkms-git`** ^{AUR} **kernel module**, providing `asus_fan`:

```
# modprobe asus_fan
```

Check if you have any control over both fans:

```
# echo 255 > /sys/devices/platform/asus_fan/hwmon/hwmon[[:print:]]*/pwm1      # Full CPU fan speed (Value: 255)
# echo 0 > /sys/devices/platform/asus_fan/hwmon/hwmon[[:print:]]*/pwm1      # CPU fan is stopped (Value: 0)
# echo 255 > /sys/devices/platform/asus_fan/hwmon/hwmon[[:print:]]*/pwm2      # Full GFX fan speed (Value: 255)
# echo 0 > /sys/devices/platform/asus_fan/hwmon/hwmon[[:print:]]*/pwm2      # GFX fan is stopped (Value: 0)
# echo 2 > /sys/devices/platform/asus_fan/hwmon/hwmon[[:print:]]*/pwm1_enable  # Change CPU fan mode to automatic
# echo 1 > /sys/devices/platform/asus_fan/hwmon/hwmon[[:print:]]*/pwm1_enable  # Change CPU fan mode to manual
# echo 2 > /sys/devices/platform/asus_fan/hwmon/hwmon[[:print:]]*/pwm2_enable  # Change GFX fan mode to automatic
# echo 1 > /sys/devices/platform/asus_fan/hwmon/hwmon[[:print:]]*/pwm2_enable  # Change GFX fan mode to manual
# cat /sys/devices/platform/asus_fan/hwmon/hwmon[[:print:]]*/temp1_input      # Display GFX temperature (will always be 0 when GFX is disabled/unused)
```

If everything works, you can [load the module at boot](#) to automate this step.

7.2 Generate configuration file with pwmconfig

If you get an error `There are no working fan sensors, all readings are 0` while generating configuration file with `pwmconfig`, open first console and execute:

```
# watch -n 1 "echo 2 > /sys/devices/platform/kernel_module/hwmon/hwmon[[:print:]]*/pwm1_enable"
```

If you use `asus_fan` kernel module and have 2nd fan, in second console:

```
# watch -n 1 "echo 2 > /sys/devices/platform/kernel_module/hwmon/hwmon[[:print:]]*/pwm2_enable"
```

And finally, in the third console:

```
# pwmconfig
```

Once you are done and the configuration file is generated, you should stop the first and second consoles. Continue with [#Fancontrol \(lm-sensors\)](#). After the configuration file is generated, you might need to manually replace PWM values with full *sysfs* paths as they are used in these steps, because *hwmon* number values might change after reboot.

7.3 Alternative method using EC registers

If the above methods do not work for you, an alternative method is to directly write to certain registers in the embedded controller (EC). Using the [EC-Probe tool](#), you can set the fan mode to one of the three fan speed modes, provided your model offers such feature in Windows.

In ASUS FX504GD model setting the fan speed to one of the three modes uses these register values:

```
# ec_probe write 0x5e 0x80 # silent mode
# ec_probe write 0x5e 0x40 # balance mode
# ec_probe write 0x5e 0xC0 # performance mode
```

Here we write to register `0x5e` that is responsible in setting the fan speed mode.

If these values do not work for you, run the *ec-probe* tool in monitor mode in Windows and try to identify which register in the EC changes value when switching through fan speed modes.

7.4 Setting thermal throttle policy

Instead of manually controlling fan speed using `asus-nb-wmi` , it is also possible to set the [thermal throttling policy](#) to have a more or less aggressive fan control policy. Possible values are `0` (default), `1` (overboost), and `2` (silent).

```
# echo $number > /sys/devices/platform/asus-nb-wmi/hwmon/hwmon[[:print:]]*/throttle_thermal_policy
```

7.5 Fan control modes on certain TUF series laptops

On certain ASUS TUF series laptops, performance and fan control modes can be changed using `Fn+F5` . The current mode can be viewed by running the following command:

```
$ cat /sys/devices/platform/asus-nb-wmi/fan_boost_mode
```

Note: On some laptops, this setting may instead be at `/sys/devices/platform/asus-nb-wmi/throttle_thermal_policy` .

You can view the value changing as you use press `Fn+F5` . 0 is "Normal Mode", 1 is "Performance Mode", 2 is most likely "Silent Mode".[\[3\]](#) It is also possible to write these values into the `fan_boost_mode` file as root and have the desired effect.

This was tested on the ASUS TUF FX504GE and ASUS TUF FX504GD models and found to be working.

You can use [tuf-fan-boost-notification-git](#) to get notifications every time the FanSpeed mode gets changed.

8 AMDGPU sysfs fan control

AMDGPU kernel driver offers fan control for graphics cards via *hwmon* in *sysfs*.

8.1 Manual fan control

To switch to manual fan control from automatic, run

```
# echo "1" > /sys/class/drm/card0/device/hwmon/hwmon0/pwm1_enable
```

Set up fan speed to e.g. 50% (100% are 255 PWM cycles, thus calculate desired fan speed percentage by multiplying its value by 2.55):

```
# echo "128" > /sys/class/drm/card0/device/hwmon/hwmon0/pwm1
```

To reset to automatic fan control, run

```
# echo "2" > /sys/class/drm/card0/device/hwmon/hwmon0/pwm1_enable
```

Warning: Resetting fan speed to auto may not work due to a driver bug and instead a restart of the driver may be required as a workaround.

8.2 Fan curves control

Newer AMD graphical cards such as RDNA3 graphical cards do not support manual fan control due to firmware limitations [\[4\]](#) . For these cases AMD provides a *fan_curve* sysfs api for controlling the fan curves, for more information on it see [\[5\]](#) .

8.3 amdgpu-fan

The [amdgpu-fan](#) package is an automated fan controller for AMDGPU-enabled video cards written in Python. It uses a "speed-matrix" to match the frequency of the fans with the temperature of the GPU, for example:

```
speed_matrix: # -[temp(*C), speed(0-100%)]
- [0, 0]
- [40, 30]
- [60, 50]
- [80, 100]
```

Launch the fan control service by [starting/enabling](#) `amdgpu-fan.service` .

8.4 amd fand-bin

Then [amd fand-bin](#) package is an native alternative to [amdgpu-fan](#) . Launch the fan control service by [starting/enabling](#) `amd fand.service` .

For this tool there are also GUI clients available: [amd guid-glow-bin](#) (Xorg) and [amd guid-wayland-bin](#) (Wayland). Before starting the client you need to [enable/start](#) `amdgui-helper.service` .

8.5 fancurve script

Not just fan controls are offered via *hwmon* in *sysfs*, but also GPU temperature reading:

```
# cat /sys/class/drm/card0/device/hwmon/hwmon0/temp1_input
```

This outputs GPU temperature in °C + three zeroes, e.g. `33000` for 33°C.

The bash script [amd gpu-fancontrol](#) by grmat offers a fully automatic fan control by using the described *sysfs hwmon* functionality. It also allows to comfortably adjust the fancurve's temperature/PWM cycles assignments and a hysteresis by offering abstracted configuration fields at the top of the script.

Tip: In order to function correctly, the script needs at least three defined temperature/PWM cycles assignments.

For safety reasons, the script sets fan control again to auto when shutting down. This may cause spinning up of fans, which can be worked around at cost of security by setting `set_fanmode 1` in the section `function reset_on_fail` .

8.5.1 Setting up fancurve script

To start the script, it is recommend to do so via **systemd** init system. This way the script's verbose output can be read via [journalctl/systemctl](#) status. For this purpose, a *.service* unit file is already included in the GitHub repository.

It may also be required to restart the script via a [root-resume.service](#) after hibernation in order to make it automatically function properly again:

```
/etc/systemd/system/root-resume.service

[Unit]
Description=Local system resume actions
After=suspend.target

[Service]
Type=simple
ExecStart=/usr/bin/systemctl restart amd gpu-fancontrol.service

[Install]
WantedBy=suspend.target
```

9 Troubleshooting

9.1 Increase the fan divisor for sensors

If *sensors* does not output the CPU fan RPM, it may be necessary to change the fan divisor.

The first line of the *sensors* output is the chipset used by the motherboard for readings of temperatures and voltages.

Create a file in `/etc/sensors.d/` :

```
/etc/sensors.d/fan-speed-control.conf

chip "coretemp-isa-*"
set fanX_div 4
```

Replacing `coretemp-isa-` with name of the chipset and `X` with the number of the CPU fan to change.

Save the file, and run as root:

```
# sensors -s
```

which will reload the configuration files.

Run `sensors` again, and check if there is an RPM readout. If not, increase the divisor to 8, 16, or 32. Your mileage may vary.

9.2 Device paths have changed in /etc/fancontrol

The enumerated *hwmon* symlinks located in `/sys/class/hwmon/` might vary in order because the kernel modules do not load in a consistent order per boot. Because of this, it may cause fancontrol to not function correctly. The error is "Configuration appears to be outdated, please run pwmconfig".

per boot. Because of this, it may cause fancontrol to not function correctly. The error is "Configuration appears to be outdated, please run pwmconfig again". [Upstream bug](#).

9.2.1 Solution

In `/etc/conf.d/lm_sensors`, there are 2 arrays that list all of the modules detected when you execute `sensors-detect`. These get loaded in by fancontrol. If the file does not exist, run `sensors-detect` as root, accepting the defaults. Open (or create) `/etc/modules-load.d/modules.conf`. Get all of the modules listed from the 2 variables in `/etc/conf.d/lm_sensors` and place them into the `/etc/modules-load.d/modules.conf` file, one module per line. Specifying them like this should make a defined order for the modules to load in, which should make the *hwmon* paths stay where they are and not change orders for every boot. If this does not work, I highly recommend finding another program to control your fans. If you cannot find any, then you could try using the alternative solution below.

9.2.2 Alternative solution: absolute paths

Using absolute file paths in fancontrol does not work by default, as its helper script `pwmconfig` is programmed to only use the *hwmon* paths to get the files. The way it does this is that it detects whether the *hwmon* path that is provided in its configuration file `/etc/fancontrol` did not change, and uses the variables `DEVNAME` and `DEVPATH` to determine such change. If your *hwmon* paths keep changing, this will prevent fancontrol from running no matter what you do. However, one can circumvent this problem. Open `/usr/bin/fancontrol`, and comment out this part of the script:

```
if ! ValidateDevices "$DEVPATH" "$DEVNAME"
then
    echo "Configuration appears to be outdated, please run pwmconfig again" >&2
    exit 1
fi
```

Note:

- Doing this may make *fancontrol* write into files you gave it in the configuration file, no matter what the file is. This can corrupt files if you provide the wrong path. Be sure that you are using the correct path for your files.
- Another thing to note is that while doing this workaround, using *pwmconfig* to create your script again will overwrite all of your absolute paths that you have configured. Therefore, it is better to manually change the old paths to the new paths if it is needed instead of using *pwmconfig*.

Commenting this out should effectively ignore the *hwmon* validation checks. You can also ignore the variables `DEVNAME` and `DEVPATH` in the configuration file as well. After this, replace all of the *hwmon* paths in the other variables with its absolute path. To make it easier, rerun `pwmconfig` with root privileges to refresh the *hwmon* devices. The *hwmon* paths in the configuration file should now point to the correct absolute paths. For each *hwmon* path, run the following command (where *N* is the enumeration of the *hwmon* path):

```
$ readlink -f /sys/class/hwmon/hwmonN/device
```

This will give you the absolute path of the device.

For example, an `/etc/fancontrol` file lists `FCTEMP5` as this:

```
FCTEMP5=hwmon2/pwml=hwmon3/templ_input
```

Executing `readlink -f /sys/class/hwmon/hwmon3/device` can, for example, output `/sys/devices/platform/coretemp.0/. cd` into this directory. If you see a `/hwmon/hwmonN/` directory, you have to do this in your *fancontrol* configuration file to replace the `hwmonN` path. From the previous example:

```
# BEFORE
FCTEMP5=hwmon2/pwml=hwmon3/templ_input
# AFTER
FCTEMP5=hwmon2/pwml=/sys/devices/platform/coretemp.0/hwmon/[:print:]]*/templ_input
```

Essentially, you must replace the *hwmon* path with the absolute path, concatenated with `/hwmon/[:print:]]*/` so that bash can catch the random enumerated *hwmon* name.

If you do not see the `/hwmon/hwmonN/` directory, then you do not have to worry about this. This means that the temperature files are in the root of the device directory. Just replace `hwmonN/` with the absolute file path. For example:

```
# BEFORE
FCTEMP5=hwmon2/pwml=hwmon3/templ_input
# AFTER
FCTEMP5=hwmon2/pwml=/sys/devices/platform/coretemp.0/templ_input
```

After replacing all of paths, *fancontrol* should work fine.

Categories: [CPU](#) | [Graphics](#)